

Scalable Technologies for Data Analysis

Pipeline Orchestration

Davide Carneiro

Mestrado em Engenharia Informática

2025/2026

Pipeline Orchestration

From "writing scripts" to "engineering systems"

- By the end of this class you should be able to
 - Distinguish between **Orchestration** and **Execution**: explain why Flyte should manage the flow while Trino/Iceberg handles the data
 - Master Flyte's core abstractions: define and implement **Tasks**, **Workflows**, and **Launch Plans** using the Flyte Python SDK
 - Design for failure: implement production-grade reliability patterns, including:
 - Retries with exponential backoff for transient errors
 - Timeouts to prevent resource leakage
 - Idempotency to ensure safe re-executions in the Lakehouse
 - Leverage strong typing: use Flyte's type system to create Data Contracts that catch integration bugs before they reach the cluster
 - Optimize with memoization: configure caching to save time and costs by skipping redundant compute steps
 - Manage environments and secrets: securely handle MinIO and Trino credentials using Secrets and manage code promotion across different Domains (Dev/Prod)

The plan

1. Intro to modern Data Platforms and scalability
2. Contract-Driven Design and Data Flow Modeling
3. Storage + table formats
4. Processing patterns + optimization
5. Orchestration + production reliability 
6. Serving: SQL/APIs/semantic layer
7. Quality + observability + cost
8. Incremental + streaming
9. ML integration
10. Governance + evolution

Previously...

- In the previous classes, we
 - Learned how to organize (from a logical perspective) a data lake
 - Discussed how to define data products and contracts
 - Saw how we can run simple one-shot data processing operations (through SQL instructions)
- Today
 - We move from scripts to workflows, to implement re-usable and observable data processing pipelines

1. Introduction

The "3 AM" Problem

- Scenario
 - You have a Python script that pulls data from MinIO, runs a heavy transformation via Trino, and updates an Iceberg table
 - It's scheduled as a cron job
 - At 03:00 AM, a transient network glitch causes the Trino connection to drop for 10 seconds
- The scripting reality
 - The script crashes
 - The data is left in a partial, corrupt state (Zombie data)
 - Downstream dashboards show wrong numbers
 - Result: You wake up to an inbox full of alerts and a morning spent on manual cleanup
- How do we build systems that fail gracefully and recover automatically?
 - We design for failure!
 - We move from writing scripts to engineering data-based systems

Common problems

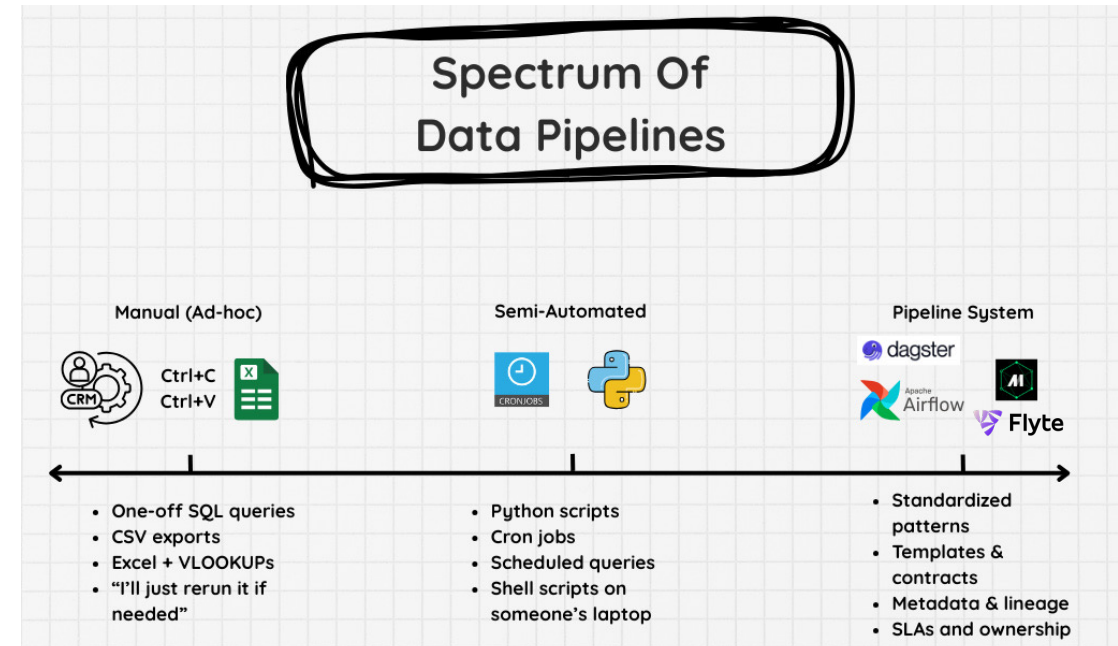
- **Monolithic Failure** – If a script with 10 steps fails at step 9, we often have to restart from step 1
 - Waste: we re-compute (and pay for) steps 1–8 again
 - Risk: steps 1–8 might not be safe to run twice (e.g. duplicate inserts into Iceberg)
- **Dependency Hell** – Scripts rely on the local environment's libraries
 - "It works on my laptop, but the production server has a different version of pandas"
- **The "Black Box" Effect** – When a script is running, you have no visibility into its internal state
 - Which specific transformation is currently bottlenecking the CPU?
 - Did the data actually arrive in MinIO, or is the script just idling?

Common problems

- **Lack of retries** – Scripts usually die on the first error. In a distributed stack (Trino + K8s), transient network errors are a fact of life, not an exception
- **No lineage or audit trail** – If a value in our Trino table looks wrong, a script won't tell which version of the code produced it or what the raw input in MinIO looked like at that exact moment
- **Concurrency and scheduling** – Scripts don't handle "What if the 2:00 AM run is still going when the 3:00 AM run starts?"
 - This leads to resource contention and data races in the Lakehouse
- **Hard-coded secrets** – Scripts often tempt developers to "temporarily" hard-code MinIO keys or database passwords, creating a massive security debt

The Workflow alternative

- Error Handling
 - Script: Manual "try/except" or crash
 - Workflow: Automatic retries and timeouts
- State
 - Script: Lost on crash
 - Workflow: Persisted (resume from failure)
- Visibility
 - Script: Print statements / Logs
 - Workflow: Visual DAG + Task-level metrics
- Environment
 - Script: Shared/Local
 - Workflow: Containerized per task
- Data Safety
 - Script: Overwrites or duplicates
 - Workflow: Idempotent by design



Production-grade expectations

- The main promise is **predictable outcomes under change and failure**
- Correctness (are results right?)
 - Outputs match business semantics and data contracts (schema, keys, grain, invariants)
 - Consistent across reruns for the same inputs (deterministic where expected)
 - Typical checks
 - Schema/contract validation, null/uniqueness constraints, referential integrity
 - Reconciliation totals, drift checks, freshness expectations
- Reliability and recovery (does it survive failures and recover safely?)
 - Handles transient failures without corrupting data (idempotency + atomic commits)
 - Supports safe retries, timeouts, circuit breakers, rollback/compensation
 - Operational capabilities: backfills (historical recompute), replay, partial reruns, “resume from checkpoint”
 - Goal: “fail fast, fail loud, recover cleanly”
- Operability (can humans run and debug it?)
 - Clear ownership and actionable signals: structured logs, metrics, alerts, runbooks, on-call friendliness
 - Fast diagnosis: “what ran?”, “with which parameters?”, “where did it fail?”, “what changed?”
 - Controllable execution: resource limits, concurrency, prioritization, maintenance windows
- Reproducibility and auditability (can we prove what happened?)
 - Every output is explainable: code version, container/image digest, parameters, inputs, upstream dependencies
 - Supports governance: lineage, approvals/promotion, access controls, retention
 - Enables regulated reporting, incident postmortems, trust in “gold” datasets

Orchestration

The Control Plane for Reliability Policies

- An orchestrator, such as Flyte, encodes policy and coordination. *It is not the compute engine*
- It usually encodes how we run safely
 - Retries/backoff, timeouts, schedules, SLAs/SLOs, concurrency limits
 - Idempotent execution patterns (e.g. partition-aware runs, rerun semantics)
 - Backfills/replays with explicit logical time
- It produces the “source of truth” for operations including execution history, parameters, artifacts, traceability, alerts hooks
- It separates concerns
 - Control plane: orchestration (policies, state, metadata about runs)
 - Data plane: compute/storage (SQL engines, object storage, table format transactions)
- In our Stack
 - Flyte defines retries/backfills and records executions
 - *Trino executes* transforms
 - Iceberg provides atomic table commits
 - MinIO stores data files

2. Reliability primitives

Idempotency

- Running the same step multiple times with the same inputs produces the same final state
- Retries, restarts, and backfills are inevitable. Idempotency prevents duplicates and corruption
- Common design patterns
 - Write target should be deterministic, keyed/determined by logical_date / partition / run parameters
 - Write intermediary processing results to staging, commit/publish final results once and after validation (quality gate e.g. expected rows)
 - Upsert/merge with stable keys: deduplicate on business key (e.g. order_id + event time)
 - Replace partition / snapshot semantics for batch windows (recompute and replace whole windows)
- Anti-patterns
 - Append on every retry
 - Using now() / random filenames to decide outputs
 - Side effects (notifications, API calls) without dedup keys
- Operational rule of thumb: if it's not idempotent, it must not be retried automatically
- In our stack
 - Iceberg helps with atomic commits / replace-partition
 - Flyte retries are safe only if tasks are idempotent

```
1 CREATE TABLE lake.staging.daily_sales_2025_10_01 AS
2 SELECT ...
3 WHERE order_ts >= TIMESTAMP '2025-10-01 00:00:00'
4 AND order_ts < TIMESTAMP '2025-10-02 00:00:00';
```

```
1 MERGE INTO orders_events t
2 USING batch_stage s
3 ON t.order_id = s.order_id
4 AND t.event_time = s.event_time
5 WHEN MATCHED THEN
6   UPDATE SET status = s.status
7 WHEN NOT MATCHED THEN
8   INSERT (order_id, status, event_time)
9   VALUES (s.order_id, s.status, s.event_time);
```

```
POST /invoices
{
  "customer_id": 42,
  "amount": 100
}
```

VS.

```
POST /invoices
{
  "customer_id": 42,
  "amount": 100,
  "idempotency_key": "invoice:order_123"
}
```

Retries

```
@task(retries=3)
def call_api(x: int) -> int:
```

- Recover from transient errors (network issues, spot interruptions, temporary service throttling)
- Retries assume idempotent side effects (or a compensation strategy)
- What to retry vs. not retry
 - Retry timeouts to external services, intermittent IO errors, ephemeral infra failures
 - Don't retry (blindly) deterministic code bugs, schema/contract violations, permanent permission errors
- Good retry policy
 - Exponential backoff (increasing times between retries) + jitter (avoid multiple simultaneous worker requests)
 - Retry budget / max attempts
 - Error classification (transient vs. permanent) to decide if and how to retry
 - Alert on exhaustion (when retries are used up)
- Key risks
 - Duplicate writes, duplicate notifications, runaway compute cost
 - "Retry storm" under downstream outages. Use circuit breakers / rate limits
- In our stack
 - Flyte policies for retries/backoff
 - Trino query retries must be guarded by idempotent output semantics

Timeouts

- A hard limit on how long a task/workflow may run before being aborted
 - Prevents “stuck forever” runs (deadlocks, hanging IO, slow downstream)
 - Protects capacity and reduces cascading failures (queue buildup)
 - Improves SLO compliance (predictable failure behavior)
- Types of timeouts to consider
 - Task execution timeout (max runtime)
 - External call timeout (HTTP/DB/object store)
 - Workflow deadline (end-to-end bound)
 - (Optional) Heartbeat/status updates for long jobs
- How to set them
 - Based on historical p95/p99 runtimes + safety margin
 - Different per environment (dev < staging < prod) and per workload class
- What must happen on timeout
 - Cancel compute cleanly, avoid partial publish, ensure cleanup of temp data
 - Emit a clear failure reason + next step
- In our stack
 - Flyte task/workflow timeouts
 - Trino query timeouts (<https://trino.io/docs/current/admin/properties-query-management.html>)
 - Iceberg commit should only happen after successful completion

```
@task(  
    retries=3,  
    timeout=timedelta(minutes=5),  
)  
def fetch_report(report_date: str) -> str:
```

```
response = requests.get(  
    "https://api.example.com/report",  
    params={"date": report_date},  
    timeout=20,  
)
```

```
from datetime import timedelta  
from flytekit import task, workflow  
  
@task(timeout=timedelta(minutes=10), retries=1)  
def extract() -> str:  
    return "raw"  
  
@task(timeout=timedelta(minutes=25), retries=1)  
def transform(data: str) -> str:  
    return data.upper()  
  
@task(timeout=timedelta(minutes=10), retries=0)  
def publish(data: str) -> str:  
    return f"published:{data}"  
  
@workflow  
def daily_wf() -> str:  
    x = extract()  
    y = transform(data=x)  
    return publish(data=y)
```

Backfills / Replays

- Recomputing historical data windows/partitions **using logical time, not “current time”**
 - Late-arriving data, bug fixes, schema changes, improved business logic
 - Enables trustworthy “gold” datasets and consistent metrics
- Backfill design principles
 - Explicit time parameters: start_date, end_date, window_size, logical_date
 - Isolation first: write to staging area/table/branch, then validate, then promote
 - Deterministic outputs: replace partition/snapshot per window
 - Throttling/concurrency control: avoid overwhelming catalogs, object storage, query engines
- Validation gates before promotion
 - Contract checks + row counts, totals, distribution/drift checks
 - (where possible) Reconciliation vs. a baseline (e.g. number of rows in sales_by_country vs. number of countries with registered clients)
- Common failure mode
 - Backfill collides with daily production runs (e.g. two workflows accessing same partition)
 - Solve with
 - Environment separation (e.g. prod / staging / dev)
 - Locks (only during publish) 
 - Partition-level ownership rules (e.g. backfill can only write on partitions before D-1)
- In our stack
 - Flyte launch plans + parameters for backfill ranges
 - Iceberg snapshot/overwrite for window rebuild
 - Trino runs transforms over historical partitions

```
-- Table
CREATE TABLE orchestration_locks (
  lock_key TEXT PRIMARY KEY,
  owner_execution_id TEXT NOT NULL,
  acquired_at TIMESTAMP NOT NULL,
  expires_at TIMESTAMP NOT NULL
);

-- Lock acquisition (might fail due to duplicate key!)
INSERT INTO orchestration_locks(lock_key, owner_execution_id, acquired_at, expires_at)
VALUES ('gold.daily_sales|2026-02-25', 'exec-123', NOW(), NOW() + INTERVAL '10 minutes');

-- Renew (if necessary)
UPDATE orchestration_locks
SET expires_at = NOW() + INTERVAL '10 minutes'
WHERE lock_key = 'gold.daily_sales|2026-02-25'
  AND owner_execution_id = 'exec-123';

-- Release
DELETE FROM orchestration_locks
WHERE lock_key = 'gold.daily_sales|2026-02-25'
  AND owner_execution_id = 'exec-123';
```


Temporal design

- Pipelines run at some time, but they usually process data for a different time
- Use a 3-time model
 - Event time (when the business event happened) e.g. order_ts inside the data
 - Logical time / data interval (the period the run is responsible for) e.g., “build the 2026-02-25 partition”
 - Execution time (when the workflow actually ran) affected by delays, retries, queueing
- Design rule: all data selection and output partitioning must be a function of logical time (data interval), not execution time
- Practical consequences
 - The pipeline must accept explicit parameters e.g. window_start, window_end (or half-open interval [start, end))
- Always define
 - Timezone policy (often: compute in UTC, report in business TZ)
 - Use safe boundaries (avoid “midnight local time” ambiguity)
- For scheduled workflows, Flyte can pass the scheduled kick-off time into a workflow input, so we don't rely on actual start time

Parameterization patterns

- Typical elements (regarding time) that are passed as parameters in a pipeline
 - Required: window_start (datetime, UTC), window_end (datetime, UTC)
 - Often useful: partition_key (string/date like YYYY-MM-DD), lookback_days (int) for late data, run_mode (daily, backfill, repair, recompute)
 - Environment + reproducibility: env (dev/staging/prod), catalog/schema/table targets (or a config object), pipeline_version / contract_version
- Good Patterns
 - Daily partition build (idempotent)
 - Read: filter source tables by [window_start, window_end]
 - Write: replace target partition for partition_key (or commit a snapshot)
 - Sliding-window repair (late data)
 - Every day, recompute D, D-1, ... D-N (small N) and merge/overwrite deterministically
 - Keeps correctness without full backfills
 - Backfill as a first-class mode
 - Same workflow, different parameter ranges
 - Concurrency throttled (e.g. limit parallelism on map task) + isolated promotion step (stage, validate, publish)
- Anti-patterns
 - "Read up to now()" (non-reproducible, makes retries/backfills inconsistent)
 - Partitioning by execution time rather than logical date
 - Local-midnight boundaries without a timezone/DST policy
 - Hidden time dependencies (e.g., "last 24h from runtime")
- Flyte operational mapping
 - Schedules are defined on Launch Plans (cron or fixed-rate), and we can bind the scheduled time to a workflow input (kickoff time arg)
 - Backfills can be driven as an orchestrated operation (e.g. pyflyte backfill creates a workflow to run a launch plan over a time window)

3. Flyte concepts

Flyte



- An open-source, cloud-native orchestrator designed to handle complex, mission-critical data and ML workflows at scale
- Developed at Lyft to solve the problem of scaling concurrent, diverse pipelines across hundreds of teams, now a Graduated project under the LF AI and Data Foundation
- Key Pillars
 - Reproducibility: every execution is **versioned** and **containerized**
 - Strong Typing: interfaces are strictly defined to prevent "silent" data failures
 - Scalability: built natively on Kubernetes, allowing it to scale from a single task to millions of concurrent executions
- The User Experience: **developers write familiar Python code** using the `@task` and `@workflow` decorators, and Flyte transforms it into a robust, distributed execution graph

Flyte mental model

- Flyte is a Kubernetes-native orchestrator: it coordinates execution of containerized steps across a cluster
- Think in three planes
 - User plane: author and submit (Flytekit / CLI / UI)
 - Control plane: validate, store metadata, orchestrate (Admin + scheduler)
 - Data plane: execute tasks (K8s pods via Propeller + plugins)
- Core entities we interact with
 - Task > Workflow > Launch Plan > Execution

Namespacing and immutability

- Flyte entities are identified by (project, domain, name, version)
- Immutability is a feature
 - Tasks/workflows are immutable once registered
 - Updates happen by re-registering a new version (old versions remain)
- Why this matters for production
 - Repeatability (“what code produced this output?”)
 - Safe rollout / rollback by selecting versions

/ flytesnacks ▾ / development ▾

Executions ▾

▾ Domain Settings

▴ All Executions in the P

- ✓ development
- staging
- production
- View All...

← SUCCEEDED flytesnacks/development/flyte_demo.workflows.flyte_workflow.training_workflow/f7ca0f90138f64d91b6d						
Domain development	Version KA9C7Plt6PyWR4BB7kyXHg==	Cluster	Time 4/3/2023 7:33:47 AM UTC	Duration 1m 13s	IAM Role default	Service A default

Tasks

- A task is the fundamental building block: a **containerized** block of compute
 - The task (our code) runs inside a container image (e.g. a Docker image) that packages
 - The code (Python/SQL scripts/binaries)
 - Its dependencies (libraries, system packages)
 - A defined runtime environment (OS userspace, filesystem layout, entrypoint)
 - When Flyte executes the task, it starts a Kubernetes Pod from that image, runs the entrypoint command, and captures outputs/logs
- So, the “unit of compute” is: one container run with specified inputs, resources, and policies (retries/timeouts, etc.).
- Tasks have
 - Inputs/outputs (strongly typed)
 - Resource requests/limits (CPU/memory), timeouts, retries, caching knobs
- Practical guideline
 - Define tasks so they're idempotent (safe to retry)
 - Make side effects explicit (writes, API calls, notifications)

```
@task
def slope(x: list[int], y: list[int]) -> float:
    sum_xy = sum([x[i] * y[i] for i in range(len(x))])
    sum_x_squared = sum([x[i] ** 2 for i in range(len(x))])
    n = len(x)
    return (n * sum_xy - sum(x) * sum(y)) / (n * sum_x_squared - sum(x) ** 2)
```

```
from flytekit import Resources

@task(requests=Resources(cpu="2", mem="100Mi"))
def light_task() -> float:
    ...

@task(requests=Resources(cpu="16", mem="16Gi"))
def heavy_task() -> float:
    ...
```

```
pyflyte run \
  https://raw.githubusercontent.com/flyteorg/flytesnacks/master/examples/basics/basics/task.py
slope --x '[-3,0,3]' --y '[7,4,-2]'
```

Task types

- At execution time, Flyte runs a task as a Kubernetes workload (typically a Pod), with resource limits, retries/timeouts, and execution tracking
- Then, there are two different types of task
- Python function task
 - We write a normal Python function, decorate it as a task, and Flytekit infers the interface (inputs/outputs) from the function signature and types
 - Strongly typed interface “for free” (from Python annotations)
 - Easy to unit test locally, refactor, and compose into workflows
 - Flytekit handles a lot of the hassle (how the function is invoked remotely, and how results are serialized) when registered/executed
- ContainerTask
 - We declare: image + command + typed inputs/outputs then Flyte runs that image and executes the command
 - It's designed to run **any language/tool** (Go/Java/R/bash/dbt/CLI tools, legacy containers, etc.) without needing Flytekit inside the container
 - Flyte provides an input directory (mounted volume) and an output directory
 - Inputs are written as files and the container writes output files back
 - Commands can template input values (e.g., {{inputs.name}}) in the ContainerTask definition

Python Task vs. Container Task

- Python tasks
 - Best for new logic as it provides clean interfaces, fast iteration
- Container tasks
 - Best for integrating existing tools and language-agnostic execution
- Both become production building blocks because Flyte's control plane applies the same policies (retries, timeouts, caching, tracking) regardless of how the task is authored

Typed dataflow

- Workflows connect tasks through data dependencies: outputs > inputs
- The goal of the type system is for data to cross process/container/language boundaries
 - So, Flyte types are serializable to a universal format (Protocol Buffers)
- Engineering benefits
 - Contract-like interfaces between steps: this goes beyond merely checking both types are DataFrames, but it looks into the actual metadata e.g. format, columns, ...
 - Safer refactors (interface changes are visible early)
 - Better caching keys

```
from flytekit import task, workflow
import pandas as pd #This is a dependency of the StructuredDataset type
from flytekit import task, workflow
```

```
@task
def create_dataset() -> pd.DataFrame:
    # This will be stored as parquet by default
    return pd.DataFrame({
        "name": ["Alice", "Bob"],
        "age": [25, 30]
    })
```

```
@task
def process_dataset(df: pd.DataFrame) -> pd.DataFrame:
    # This task can accept the dataset because the formats match
    return df[df["age"] > 25]
```

```
@workflow
def my_workflow()-> pd.DataFrame:
    df = create_dataset()
    processed = process_dataset(df=df)
    return processed
```

```
message StructuredDatasetType {
  columns = [
    { name: "name", literal_type: { simple: STRING } },
    { name: "age", literal_type: { simple: INTEGER } }
  ]
  format = "parquet"
}
```

```
from flytekit import task, workflow
import pandas as pd
```

```
@task
def create_csv_dataset() -> pd.DataFrame:
    # Let's say we explicitly set this to store as CSV
    df = pd.DataFrame({"name": ["Alice"], "age": [25]})
    return df.with_format("csv")
```

```
@task
def process_parquet_dataset(df: pd.DataFrame) -> pd.DataFrame:
    #This task expects parquet format
    df = df.with_format("parquet")
    return df
```

```
@workflow
def my_workflow()-> pd.DataFrame:
    df = create_csv_dataset()
    # This would fail type checking because formats don't match
    # CSV != Parquet
    processed = process_parquet_dataset(df=df) # Type error!
    return processed
```

Workflows

- A workflow is a declarative entity that builds a DAG from task I/O wiring
- Key concept
 - Workflow function body is evaluated at serialization/compile time, not at runtime
- Implications
 - “Normal Python control flow” isn’t always what we think
 - Runtime-dependent branching needs specific constructs (we’ll see dynamic workflows later)

```
1 import typing
2
3 from flytekit import task, workflow
4
5
6 @task
7 def t1(a: int) -> typing.NamedTuple("OutputsBC", t1_int_output=int, c=str):
8     return a + 2, "world"
9
10
11 @task
12 def t2(a: str, b: str) -> str:
13     return b + a
14
15
16 # %%
17 # You can treat the outputs of a task as you normally would a Python function. Assign the output to two variables
18 # and use them in subsequent tasks as normal. See :py:func:`flytekit.workflow`
19 @workflow
20 def my_wf(a: int, b: str) -> (int, str):
21     x, y = t1(a=a)
22     d = t2(a=y, b=b)
23     return x, d
24
25
26 # %%
27 # Execute the Workflow, simply by invoking it like a function and passing in
28 # the necessary parameters
29 #
30 # .. note::
31 #
32 # One thing to remember, currently we only support ``Keyword arguments``. So
33 # every argument should be passed in the form ``arg=value``. Failure to do so
34 # will result in an error
35 if __name__ == "__main__":
36     print(f"Running my_wf(a=50, b='hello') {my_wf(a=50, b='hello')}")
```

Workflows

- Workflows can be embedded in other workflows
- When a workflow uses another workflow as part of the execution graph, we call the inner workflow a subworkflow
 - Subworkflows are strongly typed and can be invoked just like tasks when defining the outer workflow

```
from math import sqrt
from flytekit import workflow
```

```
@task
def standard_deviation(values: List[float], mu: float) -> float:
    variance = sum([(x - mu) ** 2 for x in values])
    return sqrt(variance)
```

```
@task
def standard_scale(values: List[float], mu: float, sigma: float) -> List[float]:
    return [(x - mu) / sigma for x in values]
```

```
@workflow
def standard_scale_workflow(values: List[float]) -> List[float]:
    mu = mean(values=values)
    sigma = standard_deviation(values=values, mu=mu)
    return standard_scale(values=values, mu=mu, sigma=sigma)
```

```
import random
```

```
@task
def generate_data(num_samples: int, seed: int) -> List[float]:
    random.seed(seed)
    return [random.random() for _ in range(num_samples)]
```

```
@workflow
def workflow_with_subworkflow(num_samples: int, seed: int) -> List[float]:
    data = generate_data(num_samples=num_samples, seed=seed)
    return standard_scale_workflow(values=data)
```

```
workflow_with_subworkflow(num_samples=10, seed=3)
```

Launch Plan

- A Launch Plan is essentially the deployment + invocation configuration for a workflow
 - It templates how a workflow is launched by binding some (or all) of the workflow inputs, and optionally attaching runtime options like schedules and notifications
- A launch plan contains
 - Workflow reference (which workflow/version it launches)
 - Input bindings
 - `default_inputs`: pre-filled values that can be overridden at launch time
 - `fixed_inputs`: values that cannot be overridden (useful for enforcing invariants like `env="prod"` or fixed target tables)
 - Schedule (cron or fixed-rate) and activation state
- Default vs. custom Launch Plans
 - When we register a workflow, Flyte automatically creates a default launch plan (same name as the workflow)
 - We can define custom launch plans to run the same workflow with different defaults, schedules, notifications, and constraints
 - Launch plans are versioned, like tasks and workflows
- **Only one version** of a given launch plan (same project/domain/name) can be active at a time
 - Changing a schedule, for instance, typically implies a new launch plan version
- Workflows define the logic, Launch Plans define the operational contract for running that logic reliably in production

Scheduling

- Launch plans can **run on a schedule** using Flyte's scheduler
- Schedules can be cron-based or fixed-rate
- Critical for data pipelines
 - Flyte can pass the scheduled time (kickoff time) as a workflow input, which is useful for partition/date-driven pipelines
- In our stack, scheduled runs drive logical_date, then select Iceberg partitions, then write deterministic outputs

```
from flytekit import CronSchedule, LaunchPlan # noqa: E402

# creates a launch plan that runs every minute.
cron_lp = LaunchPlan.get_or_create(
    name="my_cron_scheduled_lp",
    workflow=date_formatter_wf,
    schedule=CronSchedule(
        # Note that the ``kickoff_time_input_arg`` matches the workflow input we defined above: kickoff_time
        # But in case you are using the AWS scheme of schedules and not using the native scheduler then switch
        schedule="*/1 * * * *", # Following schedule runs every min
        kickoff_time_input_arg="kickoff_time",
    ),
)
```

```
from datetime import timedelta # noqa: E402

from flytekit import FixedRate, LaunchPlan # noqa: E402

@task
def be_positive(name: str) -> str:
    return f"You're awesome, {name}"

@workflow
def positive_wf(name: str):
    reminder = be_positive(name=name)
    print(f"{reminder}")

fixed_rate_lp = LaunchPlan.get_or_create(
    name="my_fixed_rate_lp",
    workflow=positive_wf,
    # Note that the workflow above doesn't accept any kickoff time arguments.
    # We just omit the ``kickoff_time_input_arg`` from the FixedRate schedule invocation
    schedule=FixedRate(duration=timedelta(minutes=10)),
    fixed_inputs={"name": "you"},
)
```

Executions

- An execution is a concrete instance of running a workflow, node, or task, created either by a user launch or a schedule
- Execution IDs are unique within a project+domain and are important for traceability and audit trails
- An Execution is an immutable run record, capturing
 - The selected entity version(s) (workflow/task versions)
 - The input values used to run (e.g. logical_date, environment config)
 - The state/phase of the run and each node (success/failure/running)
- A workflow execution is made of node executions, a node usually wraps a task but can also be a subworkflow or external workflow trigger

Executions

- In Flyte Console, every execution has Graph and Nodes views. It allows to inspect
 - Execution inputs/outputs and per-node inputs/outputs
 - Logs (Kubernetes logs linked from nodes)
- Operator actions
 - Terminate/abort an execution
 - Relaunch with pre-populated inputs (optionally choosing a new version)
 - Recover a failed execution by reusing successful nodes and rerunning from the failed ones
- From the CLI / automation side
 - `flytectl get execution --details` shows node/task executions
 - Outputs can be located via an output URI stored in execution metadata
 - Programmatically, FlyteRemote can fetch execution + node outputs

The screenshot shows the Flyte Console interface for a workflow execution. The top bar indicates the execution is 'SUCCESSFUL'. Below this, a table lists the nodes in the workflow:

Task Name	Node ID	Type	Status	Start Time
get_data	n0	Python Task	SUCCEEDED	4/3/2023 7:33:47 AM UTC
process_data	n1	Python Task	SUCCEEDED	4/3/2023 7:33:47 AM UTC
train_model	n2	Python Task	SUCCEEDED	4/3/2023 7:33:47 AM UTC

On the right, a detailed view of the 'get_data' node is shown, including its logs and Kubernetes commands. The logs indicate that caching was disabled for this execution.

The screenshot shows the Flyte Console interface for a workflow execution. The top bar indicates the execution is 'SUCCESSFUL'. Below this, a graph view of the workflow is displayed, showing the sequence of tasks: 'start' -> 'say_hello' -> 'greeting_length' -> 'end'.

The screenshot shows the Flyte Console interface for a workflow execution. The top bar indicates the execution is 'SUCCESSFUL'. Below this, a timeline view of the workflow is displayed, showing the duration of each task and the overall execution time. The timeline includes a legend for 'Local time' and 'UTC'.

Parallelism primitive: Map Tasks

- Map task runs a task over a list of inputs as one workflow node, avoiding manual node explosion
- It is especially useful in data engineering tasks
 - Partition-parallel transformations (e.g., per-day or per-tenant)
 - Fan-out/fan-in patterns (compute many shards > aggregate)
- Examples
 - Executing the same code logic on multiple inputs
 - Concurrent processing of multiple data batches
 - Hyperparameter optimization
- Reliability note: map tasks amplify load so we must use concurrency limits and backpressure policies

Dynamic workflows

- A workflow whose DAG is computed at run-time is a dynamic workflow
- Used when the workflow structure depends on runtime values, which might be unknown until execution
- Why Flyte needs this
 - Normal workflows are compile-time, while dynamic workflows allow runtime expansion
- Use when there are unknown number of items/branches at runtime
- Avoid using if the graph can be static (static is easier to reason about, cache, and operate)

```
@task
def return_index(character: str) -> int:
    if character.islower():
        return ord(character) - ord("a")
    else:
        return ord(character) - ord("A")
```

```
@task
def derive_count(freq1: list[int], freq2: list[int]) -> int:
    count = 0
    for i in range(26):
        count += min(freq1[i], freq2[i])
    return count
```

```
@task
def update_list(freq_list: list[int], list_index: int) -> list[int]:
    freq_list[list_index] += 1
    return freq_list
```

```
@dynamic
def count_characters(s1: str, s2: str) -> int:
    # s1 and s2 should be accessible

    # Initialize empty lists with 26 slots each, corresponding to every alphabet (lower and upper case)
    freq1 = [0] * 26
    freq2 = [0] * 26

    # Loop through characters in s1
    for i in range(len(s1)):
        # Calculate the index for the current character in the alphabet
        index = return_index(character=s1[i])
        # Update the frequency list for s1
        freq1 = update_list(freq_list=freq1, list_index=index)
        # index and freq1 are not accessible as they are promises

    # Looping through the string s2
    for i in range(len(s2)):
        # Calculate the index for the current character in the alphabet
        index = return_index(character=s2[i])
        # Update the frequency list for s2
        freq2 = update_list(freq_list=freq2, list_index=index)
        # index and freq2 are not accessible as they are promises

    # Count the common characters between s1 and s2
    return derive_count(freq1=freq1, freq2=freq2)
```

```
@workflow
def dynamic_wf(s1: str, s2: str) -> int:
    return count_characters(s1=s1, s2=s2)
```

Reliability in Flyte

- Retries / timeouts: defined per task (and sometimes per node). They enforce failure policies centrally
- Caching: opt-in per task. Cache key includes signature + inputs. Use `cache_version` to invalidate when logic changes
- Versioning: entities are versioned/immutable. You can change workflow structure safely across versions

```
import random
from flytekit import task

@task(retries=3)
def compute_mean(data: List[float]) -> float:
    if random() < 0.05:
        raise FlyteRecoverableException("Something bad happened 🔥")
    return sum(data) / len(data)
```

```
from typing import List
from flytekit import task, workflow

@task(cache=True, cache_version="1")
def compute_mean(data: List[float]) -> float:
    return sum(data) / len(data)

@workflow
def wf(data: List[float]) -> float:
    return compute_mean(data=data)
```

```
from datetime import timedelta

@task(timeout=timedelta(hours=1))
def compute_mean(data: List[float]) -> float:
    return sum(data) / len(data)
```



4. Governing orchestration

Or how to govern the governor



Environments (Domains)

- Domains provide an abstraction (dev / staging / prod) to isolate resources and feature configuration for different deployment environments
 - We develop and deploy Flyte workflows in development, staging, and production
 - Domains isolate risk, cost, and blast radius while keeping the same code deployable everywhere
- What should differ by domain
 - Resource limits/quotas (smaller in dev, larger in prod)
 - Data targets (separate MinIO buckets/prefixes, Iceberg catalogs/schemas, Trino catalogs/schemas)
 - Schedules (off in dev, controlled in prod via active launch plans)

Secrets

```
$ kubectl create secret -n <project>--<domain> generic user-info --from-literal=user_secret=mysecret
```

- Credentials should be kept out of code, logs, and container images
- Flyte supports a native Secret construct to request/access secrets in tasks
- Two mount patterns
 - ENV var secret (used by default if not specified)
 - FILE mount secret (often safer, apps read from a mounted path)
- Secrets consists of group, key, and mounting_requirement arguments
 - A secret group can have multiple secrets associated with it
 - A secret can have multiple keys
- Operational best practices
 - Least privilege per task/service account
 - Rotate secrets, avoid long-lived static keys
 - Never print secrets, scrub logs on failure paths
- Under the hood secrets are managed by Kubernetes (or integrated secret stores) and injected into task pods at runtime

```
$ kubectl create secret generic user-info \
  --from-literal=user_secret=mysecret \
  --from-literal=username=my_username \
  --from-literal=password=my_password
```

```
import os
from typing import Tuple

import flytekit
from flytekit import Secret, task, workflow
from flytekit.testing import SecretsManager

secret = Secret(
    group="<SECRET_GROUP>",
    key="<SECRET_KEY>",
    mount_requirement=Secret.MountType.ENV_VAR,
)
```

```
@task(
    secret_requests=[
        Secret(key=USERNAME_SECRET, group=SECRET_GROUP),
        Secret(key=PASSWORD_SECRET, group=SECRET_GROUP),
    ]
)
def user_info_task() -> Tuple[str, str]:
    context = flytekit.current_context()
    secret_username = context.secrets.get(SECRET_GROUP, USERNAME_SECRET)
    secret_pwd = context.secrets.get(SECRET_GROUP, PASSWORD_SECRET)
    print(f"{secret_username}={secret_pwd}")
    return secret_username, secret_pwd
```

Versioning

- Flyte entities are versioned and immutable: a specific workflow/task version cannot be modified after registration
 - Enables reproducibility (“what code ran?”)
 - Supports safe rollout/rollback (switch which version you execute)
 - Allows changing workflow structure between versions without breaking historical runs
- Production pattern
 - Treat container image digest + Flyte entity version as a single deployable unit
 - Use Launch Plans to select the “current” version for schedules/ops (promote staging > prod)
- Common pitfall: changing business logic without updating versioning/contract assumptions
 - This breaks downstream expectations

5. Hello Flyte

Hello Flyte

- Copy the hello_flyte project, inspect it, and install dependencies
- Set Trino connection variables
- Run the workflow
- Verify the result in Trino
- Also open Flyte UI and inspect:
 - the execution graph
 - inputs (logical_date, table names)
 - node logs (basic traceability)



```
cd flyte_intro
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

```
cd flyte_intro
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

```
export TRINO_HOST=localhost
export TRINO_PORT=8080
export TRINO_USER=tead
```

```
$env:TRINO_HOST="localhost"
$env:TRINO_PORT="8080"
$env:TRINO_USER="tead"
```

```
pyflyte run simple_pipeline.py daily_event_counts \
  --logical_date 2026-02-25 \
  --source_table iceberg.<your_schema>.<your_bronze_table> \
  --target_table iceberg.<your_schema>.gold_daily_event_counts
```

```
SELECT * FROM "iceberg.<your_schema>.gold_daily_event_counts$partitions"
ORDER BY total_size DESC;
```

```
SELECT *
FROM iceberg.<your_schema>.gold_daily_event_counts
WHERE day = DATE '2026-02-25'
ORDER BY event_count DESC;
```

Scalable Technologies for Data Analysis

Pipeline Orchestration

Davide Carneiro

Mestrado em Engenharia Informática

2025/2026